traditional approaches. Other studies have found a correlation between Test-Driven Development practices, reduced bug counts,[5] and improved code quality.[6] Although it's certainly possible to write high-quality code without practicing techniques such as Test-Driven Development and Clean Coding, teams that value good development practices do seem to succeed in delivering high-quality code more often.

But building high-quality software isn't in itself enough to guarantee a successful project. The software must also benefit its users and business stakeholders.

1.2.2 Building the right software

Software is never developed in a vacuum. Software projects are part of a broader business strategy, and they need to be aligned with business goals if they're to be beneficial to the organization. At the end of the day, the software solution you deliver needs to help users achieve their goals more effectively. Any effort that doesn't contribute to this end is waste.

In practice, there's often a lot of waste. In many projects, time and money are spent building features that are never used or that provide only marginal value to the business. According to the Standish Group's CHAOS studies,[7] on average some 45% of the features delivered into production are never used. Even apparently predictable projects, such as migrating software from a mainframe system onto a more modern platform, have their share of features that need updating or that are no longer necessary. When you don't fully understand the goals that your client is trying to achieve, it's very easy to deliver perfectly functional, well-written features that are of little use to the end user.

On the other hand, many software projects end up delivering little or no real business value. Not only do they deliver features that are of little use to the business, but they fail to even deliver the minimum capabilities that would make the projects viable.

The consequences of not building it right, and not building the right thing

The impact of poorly understood requirements and poor code realization isn't just a theoretical concept or a "nice to have;" on the contrary, it's often painfully concrete. In December 2007, the Queensland Health Department kicked off work on a new payroll system for its 85,000 employees. The initial budget for the project was around $6 million, with a delivery date of August 2008.

---

[5] See, for example, Nachiappan Nagappan, E. Michael Maximilien, Thirumalesh Bhat, and Laurie Williams, "Realizing quality improvement through test driven development: results and experiences of four industrial teams," http://research.microsoft.com/en-us/groups/ese/nagappan_tdd.pdf.

[6] Rod Hilton, "Quantitatively Evaluating Test-Driven Development by Applying Object-Oriented Quality Metrics to Open Source Projects" (PhD thesis, Regis University, 2009), http://www.rodhilton.com/files/tdd_thesis.pdf.

[7] The Standish Group's *CHAOS Report 2002* reported a value of 45% and I've seen more recent internal studies where the figure is around 50%.